THE AI GAME STUDIO · ENGINE · TEMPLATES · RAINBOW RUN

CLAYSTONE

A determinism-first 2D game engine — and the templates & games built on it.

One person, one press, a shipped and self-improving game. This deck walks the engine that makes that possible, the templates that make it fast, and Rainbow Run — the flagship that proves it — end to end.

BUILT & AI-EVALUATED WITH CLAUDE CODE · 2026

THE STRUCTURE OF THIS DECK

Six acts, 31 slides

I · The problem & the thesis

Why AI-built games hit a wall — and the single press that gets past it.

II · Claystone, feature by feature

The determinism lock-in, the architecture, and every subsystem.

III · Claystone vs Phaser

An honest, sourced comparison — and why the design works.

IV · The templates

Hit "new game" and go — what ships in the box, and the pipeline.

V · Rainbow Run

25 worlds, the notes→fixes loop, and the Claystone migration.

VI · The whole system

How engine, templates and games compound into one studio.

Every number and quote in this deck is drawn from the live repositories — **claystone-engine**, **game-engine**, the templates, and **rainbow-run**.

THE PROBLEM

To let an AI build games, an AI must test them — and that breaks on flakiness

The studio's promise is that **one autopilot must beat every level at zero deaths** before it ships. That gate is only meaningful if the game runs **the same way every time**.

But a normal engine runs its own clock, tied to `requestAnimationFrame` and wall-time. A single slow frame integrates a huge delta and flings the player across the world on one stale decision. Replays diverge; the gate becomes noise.

> *"An autopilot must win every level deterministically; flakiness would make the gate meaningless."*

So the real engineering problem isn't graphics. It's **reproducibility** — a loop you can drive by hand, frame by frame, and get bit-identical results, forever.

THE THESIS

One press → a shipped, self-improving game

Hit **"new game"** and go. A clean base is cloned, rebranded, given a GitHub repo, registered with a mission-control hub, and issue-tracked — in a single command. Then an autonomous pipeline designs levels, an **AI player gates them at 0 deaths**, art and music are generated, and it deploys itself.

Once live, players leave in-game notes; those become issues; the agent fixes them, re-gates, and writes the "why" into an in-game build diary. The game gets better on its own.

Everything in that loop — the gate, the recorder, the headless design scans, the ghost-run editor — **stands on one foundation: a deterministic engine.** That engine is Claystone.

MEET THE ENGINE

Claystone 🧱

> *"A determinism-first, custom 2D game engine to replace Phaser under the AI game studio SDK. Zero dependencies. Runs headless in Node — which is how the determinism gate is tested — and in the browser via a Canvas2D backend."*

Its own framing: replacing Phaser is **not primarily a graphics project — it's a determinism & integration project**. Commodity 2D-engine features **plus one special thing**: the deterministic, steppable, headless loop. That's the real lock-in.

0	111	3,862	~20×	v0.1
RUNTIME DEPENDENCIES	TESTS, ALL GREEN	LINES OF ENGINE SOURCE	FASTER HEADLESS	AND ALREADY SHIPPING GAMES

THE CROWN JEWEL · ISSUE #1

Take over the clock. Step time by hand.

The SDK sleeps the RAF driver and advances the simulation **one fixed 1/60s frame at a time, by hand**. Combined with a **seeded RNG** — never `Math.random` — a run is **bit-for-bit reproducible, forever**.

```
game.rec.begin();   // sleep the loop, take the clock
game.rec.step(1);   // advance exactly one 1/60s frame
const s = game.snapshot();
// same inputs ⇒ same s, always
```

Everything else — the 0-death gate, the recorder, headless scans, the ghost-run editor — stands on this one property.

Two mechanisms, one guarantee

- **Seeded mulberry32 RNG** — "same seed + same draws ⇒ the same sequence, forever."
- **Hand-driven fixed timestep** — the clock is the SDK's, not the browser's.
- **Fixed update order** — "change it and you change every replay."
- **No `Math.random` in `src/`** — enforced by a test that walks the tree.

Proven: two independent gate runs end identical; a 360-frame recording replays byte-for-byte.

ARCHITECTURE

The studio is a stack — each tier needs only the one beneath it

Seven load-bearing tiers

- **Foundations** — the method + a working base, inherited whole.
- **Platform** — server, volume store, Gemini, Lyria, Railway.
- **Engine** — Phaser 4 or Claystone, driven through the Studio SDK.
- **Content & authoring** — art pipeline, per-world music, level builder.
- **Evaluation** — the 0-death gate, felt-fun, recorder, vision judge.
- **Feedback loop** — in-game notes → issues → build diary.
- **Publish & ship** — YouTube shorts + Railway auto-deploy.

Inside Claystone (`src/`)

- **core** — game loop, RNG, scene, events, timers
- **physics** — AABB world + bodies
- **render** — null · canvas2D · webgl · pixi
- **display** — sprites, shapes, tiles, text
- **anim** — tweens + frame animation
- **fx** — camera, particles, lights, shadows, env
- **input / audio / scale**
- **editor** — the proof-carrying AI editor

One `Studio.*` SDK sits on top — the same call surface a Phaser game uses.

FEATURE · THE LOOP

A fixed-timestep heartbeat + a seeded stream

Deterministic loop

- Fixed `DT = 1000/60` with an accumulator and a `250ms` spike-clamp.
- Live play uses RAF; the gate and recorder **hand-drive** the same `step()`.
- Default 960×540, default seed `0x1234abcd`.

Seeded RNG

- **mulberry32** 32-bit PRNG: `u32 · float · range · int · chance · pick`.
- `snapshot / restore` — the RNG is part of savable state.
- "The same seed + same number of draws ⇒ the same sequence, forever. This is half of what makes runs bit-exact."

The other half is discipline: a documented **fixed update order** (game update before physics) — a real Rainbow Run bug traced to a sub-frame phase offset that compounded into a missed landing. Order is now test-locked.

FEATURE · PHYSICS

Custom AABB — exactly the subset the games use, deterministically

Arcade-style axis-aligned collision, reimplemented for bit-exact separation — not a general rigid-body sim, by design.

- **Swept resolution** via previous-position, axis-separated.
- **Per-edge `checkCollision` toggles** — one-way platforms: jump up through, land on top.
- Static & dynamic groups; registered colliders vs overlaps.
- World bounds; `blocked / touching` flags; direct-velocity model.
- Overlap callbacks drive win/death without touching the sim step.
- **Proven:** gravity+floor, wall-blocking, deterministic separation — all test-green.

Deliberately **not** built: tilemaps (levels are data), Matter/Box2D, gamepad, bitmap fonts. The explainer estimates the loop + AABB physics are **~70% of the risk** in any engine replacement — so Claystone owns exactly those.

FEATURE · RENDERING

Three renderers, one identical simulation underneath

null · headless

The test/gate path. Rendering stubbed out → **~20× faster** and byte-identical to a rendered run.

canvas2D · default

What plays in the browser. Includes `readPixels` / `isAllBlack` so the gate can prove a frame isn't blank.

pixi · rich

Vendored **PixiJS v8** (WebGL), lazily imported in-browser only. GPU bloom, grade, CRT, lights.

The invariant: the simulation never touches a renderer. Headless equals rendered, byte-for-byte — so you can evaluate a game at 20× speed with total confidence it matches what a player sees. Pick a backend with `?r=`; pixi falls back to canvas when GL is missing.

FEATURE · RICH RENDERING

Cinematic FX that physically cannot touch the sim

An entire "sim-blind" FX layer reads display objects and the frame counter but **never writes simulation state**. Beauty with zero risk to the gate.

- **Camera** — follow, shake, flash, fade, bounds
- **Particles** — pooled, zero-allocation emitters + trails
- **Post-FX** — filter stacks & graded one-shots
- **Lights** — ambient / point / spot + normal maps
- **Shadows** — raycast volumes + blob drops
- **Env** — one declarative atmosphere block

One line of weather

```
Studio.Env.set(scene, {
  grade:'dusk', godrays:[{angle:30}],
  weather:{kind:'rain', density:2},
  lightning:{period:420, jitterSeed:5},
  fog:[{y0:430,y1:500}], water:{line:520}
});
```

Time-varying effects advance from the frame counter; jitter uses per-emitter seeded RNGs — so FX can never shift the sim's stream.

Graceful degrade is a rule, not luck: **"geometry-ish atmosphere degrades, filter-ish polish disappears"** — full on pixi, softened on canvas, inert headless. The gate compares sim output: byte-identical with any FX declared.

FEATURE · LEVEL DSL

Levels are data, not tilemaps

A level is a plain spec that `Studio.Level.build()` turns into a playable world. No paint, no tile editor.

```
const SPEC = {
  groundY: 500,
  ground: [[0,1380],[1460,2000]],   // pit 1380→1460
  platforms: [[1080,360,120,16, true]], // one-way
  coins: [[300,470],[640,470]],
  goal: [1820,420,40,80],
};
```

Machine-readable Materials

Named surfaces carry their own physics + intent: **SOLID** **ICE** **MUD** **CLOUD** **LAVA** **THORN** **VINE** each with friction, a `deadly` flag, and a colour.

Levels are AI-completable by construction — because "deadly" and "footing" are **data, not pixels**. An agent can read a level's danger and reason about it directly.

FEATURE · THE AI EDITOR

Proof-carrying editing — every change plays itself instantly

Because the loop is steppable and bit-exact, every edit triggers an **instant, invisible playthrough whose result is a fact**, not a guess.

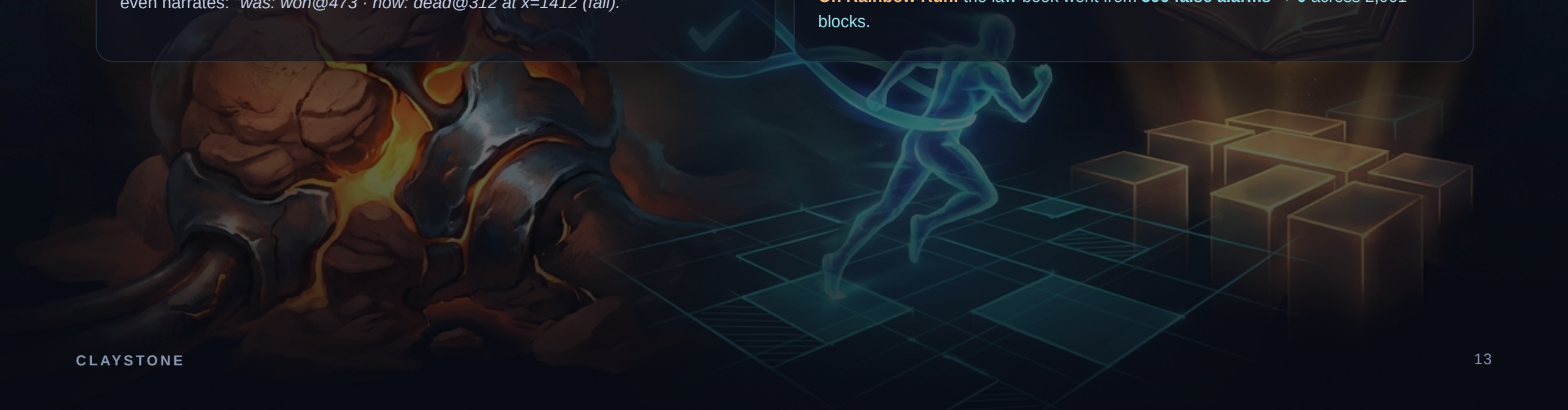
```
session.place(block)
// → { ok, errors, violations, ghost }
//   ghost = { trail, outcome, hash, cached }
```

An exact cache keyed by spec hash means unchanged levels cost nothing. The diff even narrates: *"was: won@473 · now: dead@312 at x=1412 (fall)."*

A machine-checkable law-book

- Placement laws derive from the real **jump envelope** — `maxGap`, `maxRise` from run/jump/gravity constants.
- Each violation ships a `{law, msg, fix}` — a suggested legal placement.
- "Proof rides along with every edit" — a session can never be left unproven.

On Rainbow Run: the law-book went from **596 false alarms → 0** across 2,061 blocks.



FEATURE · THE EDITOR · "THE GAME IS THE EDITOR"

Freeze the frame, drag a part from the toybox, drop it in

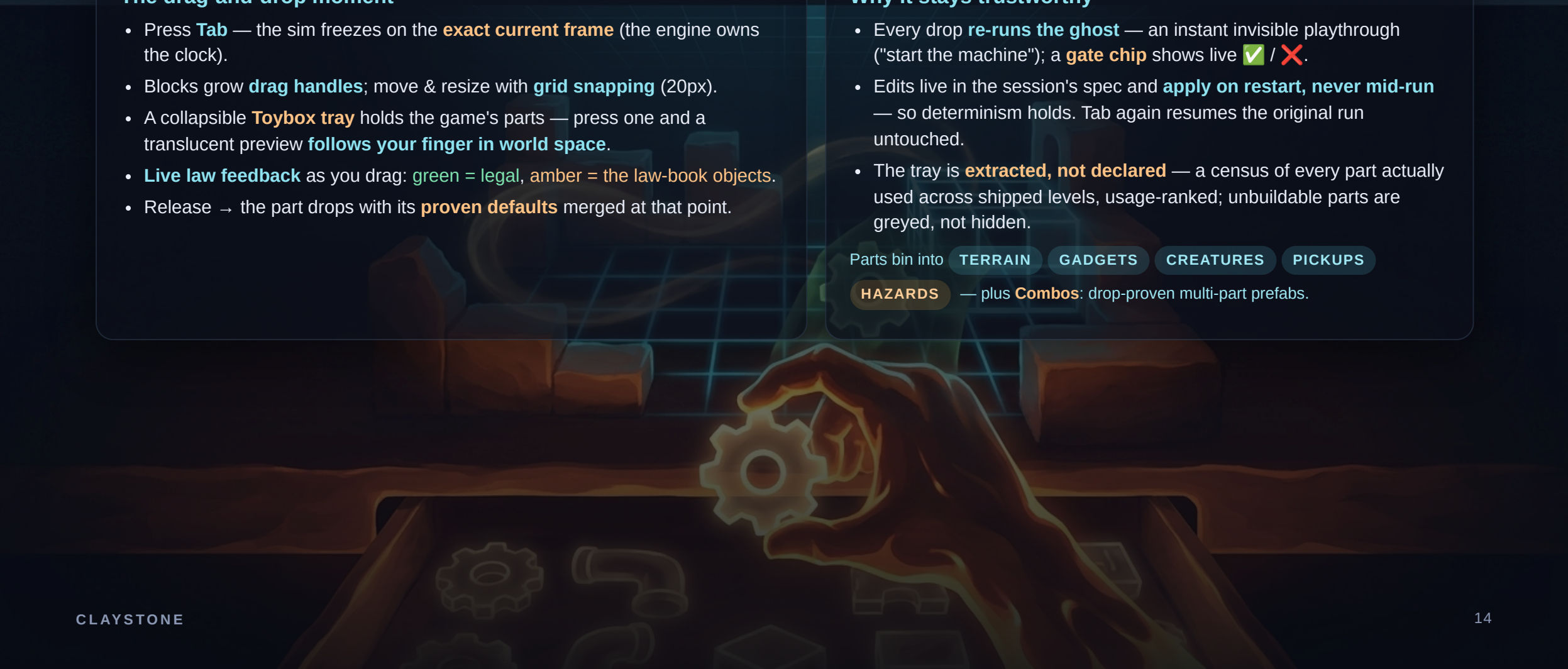
The drag-and-drop moment

- Press **Tab** — the sim freezes on the **exact current frame** (the engine owns the clock).
- Blocks grow **drag handles**; move & resize with **grid snapping** (20px).
- A collapsible **Toybox tray** holds the game's parts — press one and a translucent preview **follows your finger in world space**.
- **Live law feedback** as you drag: green = legal, amber = the law-book objects.
- Release → the part drops with its **proven defaults** merged at that point.

Why it stays trustworthy

- Every drop **re-runs the ghost** — an instant invisible playthrough ("start the machine"); a **gate chip** shows live ✅ / ❌.
- Edits live in the session's spec and **apply on restart, never mid-run** — so determinism holds. Tab again resumes the original run untouched.
- The tray is **extracted, not declared** — a census of every part actually used across shipped levels, usage-ranked; unbuildable parts are greyed, not hidden.

Parts bin into TERRAIN GADGETS CREATURES PICKUPS HAZARDS — plus **Combos**: drop-proven multi-part prefabs.



FEATURE · AI EVALUATION

A robot QA team the engine ships with

`game.gate(3000)` resets, turns on the autopilot, and steps to win / death / timeout — returning `{ won, dead, frame }` **bit-exact**. The reference level is beaten at frame 473.

And the gate actually discriminates. Give the autopilot its jump and it wins with zero deaths; take the jump away and it falls in the pit and reports `dead` at the right x-position. A gate that always passes proves nothing — this one doesn't.

0	473	~20×	4
DEATHS — THE SHIP BAR	FRAMES TO BEAT THE REFERENCE	FASTER, RENDERING STUBBED	LENSES: WIN · FEEL · RECORD · JUDGE

WHY THE BAR IS 0 DEATHS

One clean run is the studio's whole safety net

1 · Proof it's solvable & fair

An AI placing levels at scale has no gut feel. A 0-death run is a **machine-checkable proof** a real, executable path exists — no impossible jumps, no unreachable goals.

2 · Binary — so it can be a gate

"Fun" is fuzzy; **pass/fail** is not. Only an unambiguous bar can be wired into CI as a hard ship rule the machine enforces every time.

3 · It lets one person ship

It's a **robot QA team you trust more than yourself** — running on every level, every change, so nobody replays 25 worlds by hand.

4 · Only real because it's deterministic

A pass is worthless if the next run might die from a timing fluke. Claystone's bit-exact loop makes "0 deaths" a **durable fact, not a lucky roll**.

5 · It actually discriminates

Give the bot its jump → it wins clean. Take the jump away → it falls in the pit and reports `dead`. A gate that only ever says "pass" proves nothing — this one doesn't.

A floor, not a ceiling

0-death proves a level is **beatable and fair** — not that it's *fun*. That's a separate axis (the felt-fun eval). This is the non-negotiable baseline everything else builds on.

BEYOND THE GATE

0-death is one lens. The playtester has several.

The core evaluators

- **wincheck** — the 0-death gate; the ship bar.
- **feel** — a felt-fun score (engagement, dynamics, arc, flow) that catches "beatable but boring."
- **record** — the deterministic recorder; bit-identical replays and video.
- **judge** — a vision art-director that eyeballs frames for look & readability.
- **game-diff / feel-judge** — prove a new game is genuinely *its own game*, not a re-skin.

The reproducibility & reality checks

- **determinism check** — two fresh runs must end byte-identical on x/y/vx/vy/frame/deaths/won/coins.
- **parity gate** — Claystone runs matched route-for-route against recorded Phaser golden traces (±10% frames, ≤40px route drift).
- **non-black render** — the frame actually drew something, on webgl *and* canvas.
- **analytics-calibrated eval** — real player death-hotspots feed back so the bot is tuned to where humans actually struggle.

The perfect autopilot once "shipped the NEXT button broken twice" — so the eval doesn't stop at *can the bot win*. It asks **does it render, is it fun, does it reproduce, and does it match how real people play**.

FEATURE · FOOTPRINT

Zero-dependency — and it stays that way

The zero-dep amendment

- **No dependencies, no devDependencies** — tests run on Node's built-in `node --test`.
- The **simulation is zero-dependency, forever.**
- Rich visuals may use vendored, pinned, MIT-licensed libs — **lazy-imported in the browser only**, never on the headless/test path (enforced by a static-import sweep).
- No CDN at runtime, ever.

What that weighs

A Claystone game ships as a **static site** — readable ES modules, no bundle, no server needed to play.

~0kb
THIRD-PARTY ON THE PLAY PATH

352kb
TOTAL READABLE ES SOURCE

For contrast, the Phaser build the other games ship is a fixed **1.13 MB** on every page load.

One table, eight dimensions

DIMENSION	CLAYSTONE 🧱	PHASER
Engine bundle	**~0 KB** third-party on the play path · 352 KB readable ES modules · zero runtime deps	**1.13 MB** `phaser.min.js` on every page
Determinism	**Native** — fixed 1/60s hand-stepped loop + seeded mulberry32, bit-exact	**Retrofitted** — SDK sleeps Phaser's clock + delta-clamp workaround
AI gate / replay / scan	**Built-in** · ~20× faster headless · proof-carrying ghost-run editor	Works — but only because the SDK forces determinism onto it
Rendering	null / canvas / optional pixi-WebGL · sim-blind FX · graceful degrade	Canvas/WebGL, mature — but GPU post-FX avoided (crashes in containers)
Physics	Custom AABB — one-way, swept, deterministic	Arcade AABB · Matter/Box2D available but unused here
Build step	**No-build** — static ES modules, served as-is	**No-build** too — loaded as a script-tag global
Maturity / ecosystem	v0.1.0, private, single-studio, purpose-built	**Mature** — 10+ yrs, huge community, deep docs
License	Proprietary (vendored FX libs are MIT)	**MIT**, open source

CLAYSTONE VS PHASER · THE HONEST READ

Different tools. Pick for the job.

Claystone wins when…

- You need **reproducible 0-death gates**, replays and headless design scans.
- An **AI builds and evaluates** the game — determinism is native, not bolted on.
- Footprint matters — a static, ~0-dep site that loads instantly.
- You want **full control** and a drop-in `Studio.*` seam to port games unchanged.

Phaser wins when…

- **Maturity & ecosystem** lead — thousands of examples, plugins, Stack Overflow.
- A **human developer** wants the gentle on-ramp and vast public API.
- You want **MIT open-source** governance and community ownership.
- You need engine features Claystone deliberately skips (tilemaps, Matter/Box2D).

Kept honest: the repos contain no head-to-head fps benchmark, and Claystone is pre-1.0. Its documented edges are **determinism, footprint and AI-eval throughput** — not proven runtime-rendering superiority, and not ecosystem maturity, where Phaser clearly leads.

WHY THIS DESIGN WORKS

Five choices that reinforce each other

1 · Determinism is the keystone

Reproducibility isn't a feature — it's the foundation the gate, recorder, scans and editor all rest on. Get it right once, everything above becomes possible.

2 · Headless-first is honest & fast

Because headless equals rendered byte-for-byte, evaluation runs ~20× faster *and* tells the truth. Speed and fidelity stop being a trade-off.

3 · Data, not paint

Levels-as-data with typed Materials make a world *readable* to an AI — danger and footing are facts it can reason over, so it can build and fix levels.

4 · Beauty without risk

The sim-blind FX layer means rich weather, lights and post-FX can never destabilize the gate. Polish is free of consequence.

5 · A shared seam

The same `Studio.*` SDK and 0-death contract span Phaser and Claystone — so a game is a drop-in port, not a rewrite.

The proof

Golden-trace parity: Claystone's runs are checked route-for-route against recorded Phaser traces — migration fidelity you can measure, not hope for.

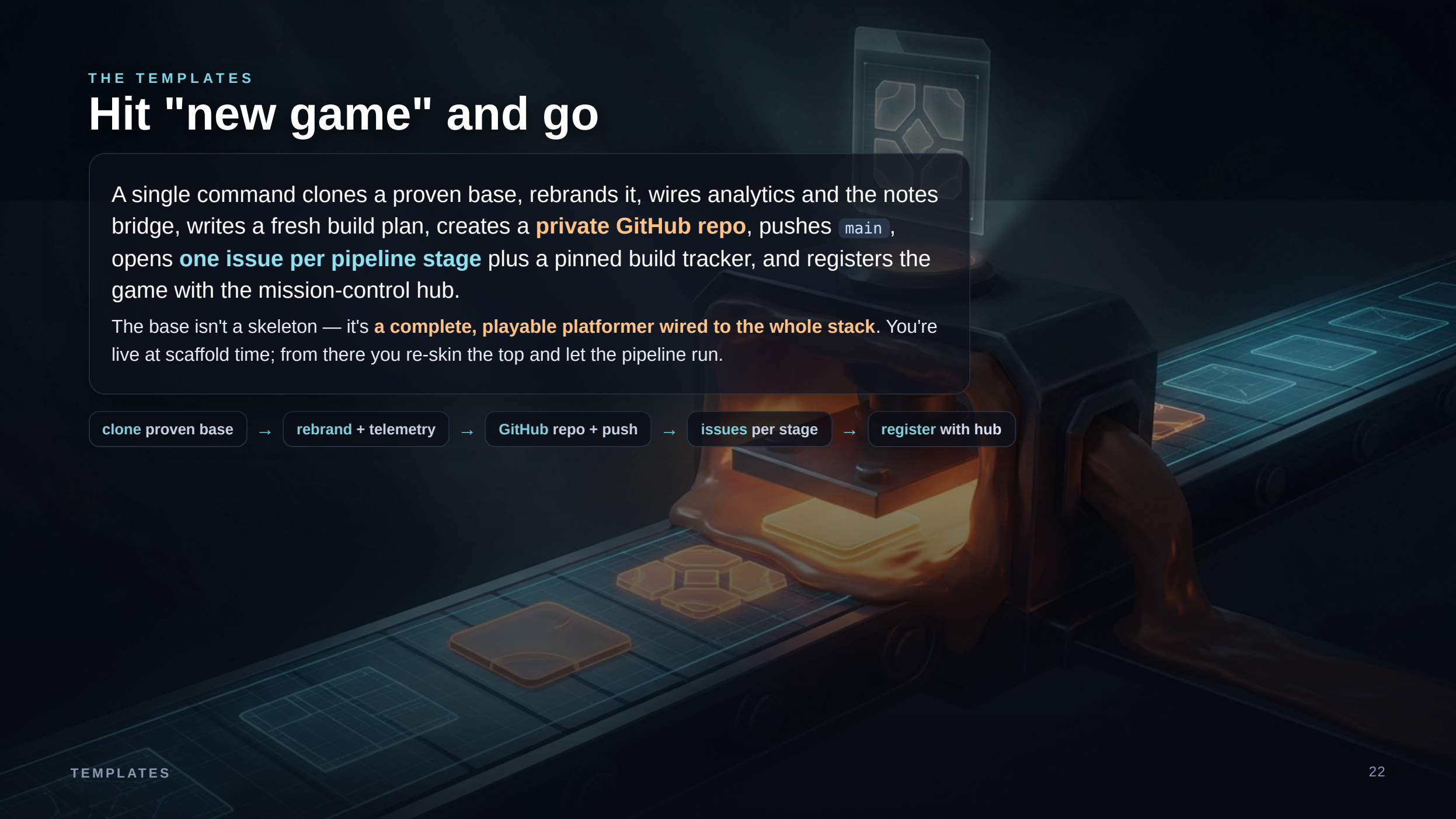
THE TEMPLATES

Hit "new game" and go

A single command clones a proven base, rebrands it, wires analytics and the notes bridge, writes a fresh build plan, creates a **private GitHub repo**, pushes `main`, opens **one issue per pipeline stage** plus a pinned build tracker, and registers the game with the mission-control hub.

The base isn't a skeleton — it's **a complete, playable platformer wired to the whole stack**. You're live at scaffold time; from there you re-skin the top and let the pipeline run.

clone proven base → rebrand + telemetry → GitHub repo + push → issues per stage → register with hub



THE TEMPLATES

What ships in the box

Build & design

- **Level DSL** — `Studio.Level.build`
- **Reward model** per-entity
- **Autopilot** — the gate's bot
- **Merge / checkpoints**
- **Art pipeline** — Nano Banana Pro
- **Music** — Lyria per world
- **Canvas level builder**
- **Chat brain** (Phaser base)

Evaluate & ship

- **0-death gate** — wincheck
- **Felt-fun** — feel model + judge
- **Deterministic recorder**
- **Vision art-director**
- **Notes → issues** bridge
- **In-game build diary**
- **Shorts / video pipeline**
- **Railway deploy recipe**

Most tooling lives **once** at the engine level and is inherited, not copied — so a fix to the gate or recorder improves every game at once. A curated set of `.claude` skills (`make-game · gate · design-pass · merge · trailer · add-character` …) drive it all.

THE TEMPLATES

One base, three flavors

studio-game-template

THE DEFAULT

Phaser 4 · arcade physics · webgl+canvas. Deliberately minimal — hub contracts + static game, one dependency. Ships with the **deterministic 0-death eval**.

game-template

THE FULLY-LOADED ONE

Phaser 3, the richest tool tree — feel eval, recorder, showcase, plus an **Anthropic chat brain** baked into the server. The "jazz" lineage.

claystone-platformer

THE DETERMINISM BUILD

Custom engine · zero-dependency static site · same game runs headless in Node under the gate. `--engine claystone`.

The seam is the point: **the same `Studio.*` SDK and the same 0-death gate contract span all three**. Phaser stays the default; Claystone is the opt-in determinism-first alternative — and a game moves between them as a port, not a rewrite.

Concept in, shipped game out

One runner executes the arc headlessly, with checkpoints, auto-retry and stop-on-fail — resumable from where it left off, with a live build board.

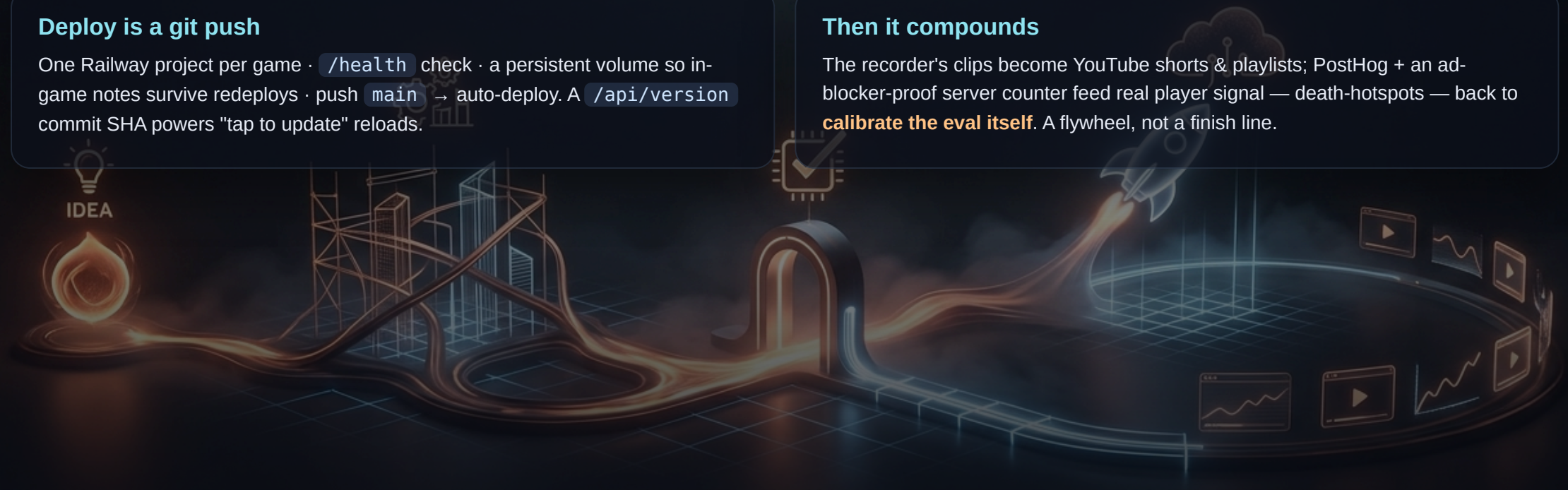
scaffold · identity · levels · gate · feel · art · music · deploy · videos · shorts · loop

Deploy is a git push

One Railway project per game · `/health` check · a persistent volume so in-game notes survive redeploys · push `main` → auto-deploy. A `/api/version` commit SHA powers "tap to update" reloads.

Then it compounds

The recorder's clips become YouTube shorts & playlists; PostHog + an ad-blocker-proof server counter feed real player signal — death-hotspots — back to **calibrate the eval itself**. A flywheel, not a finish line.



THE FLAGSHIP

Rainbow Run

"Run the rainbows with Labooboo & Siya — the weather shifts as you go." A bright, children's-marker-art **run · jump platformer** where every level is a distinct biome with its own signature traversal verb.

The story: a creeping grey hush called **THE GREYOUT** is swallowing the rainbow roads. **Labooboo** runs the ground, **Siya the unicorn** runs the sky — racing the rainbows to relight them before the last colour fades. The antagonist isn't a boss to stomp; it's the colourless weather itself, felt as the world turns.

Live: `rainbow-run-production.up.railway.app` — scaffolded from the studio template, then grown for months.

RAINBOW RUN · SCOPE

25 worlds, 25 biomes — grown 5 → 20 → 25

25	11	18	7+13	5
WORLDS / LEVELS	BOSSES, 11 THEMES	POWER-UPS, ONE SPINE	ENEMY KINDS + BEHAVIORS	PLAYABLE SKINS

Each world is a distinct biome + a single signature mechanic, built on a fractal length ladder (4000→10800px) with its own personality, palette, fauna and art set:

Sunbeam Meadow · Drizzle Dunes · Rainbow Bridge · Frostspark Peaks · Thunder Heights · Bouncy Bog · Windy Cliffs · Cavern Depths · Lava Foundry · Clockwork Towers · Candy Kingdom · Aqua Reef · Haunted Hollow · Desert Mirage · Sky Citadel · Crystal Caves · Jungle Canopy · Volcano Rim · Frozen Tundra · Rainbow Cosmos · Prism Hollow · The Gearworks · Sky Rails · Sunken Vault · The Everstorm

Signature mechanics span conveyors, bounce/dash pads, crumble bridges, fog + flicker, water, movers, chase, timed doors, portals, rotating gears, pendulum blades, ziplines, geysers, whirlpools and rhythm pads — variety kept by hand, so no two screens feel the same.

Every sprite and score, generated

Art · Nano Banana Pro

- **Gemini 3 Pro Image** (`gemini-3-pro-image-preview`) — this deck's art too.
- **Reference-conditioned** sheets keep a character on-model across many poses.
- Corner flood-fill background removal — no chroma-key.
- Batch-API path ~50% cheaper for the ~640-asset build.

`assets/manifest.json` is the source of truth: **242 assets**, 26 backdrops with 3-layer parallax.

Music · Lyria

- A distinct track per world + a title theme.
- **11 boss themes** — Storm King, Bog Warden, Gale Wyvern, the VOID KING…
- **38 tracks** in all, standardized on Lyria 3.

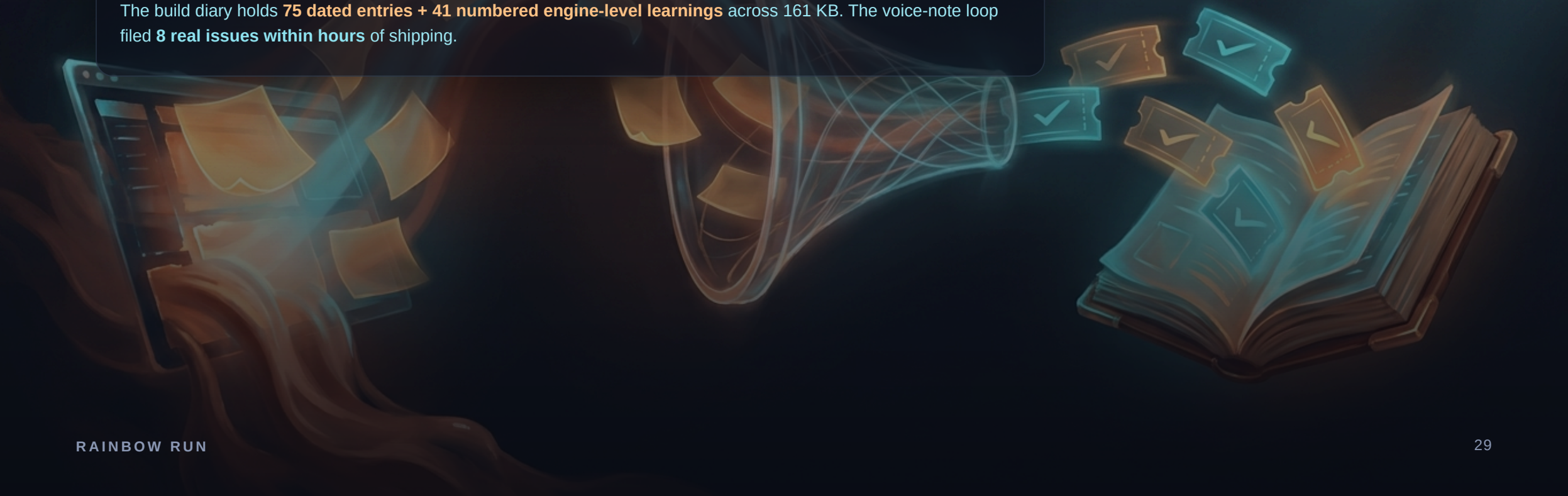
Art and music aren't hand-drawn assets bolted on — they're **generated content the pipeline produces on demand**, versioned like code.

"I felt something" → it's fixed → it's on the record

📝 / 🎙️ in-game note → /api/notes on volume → GitHub issue (auto) → /fix-notes → gate → ✅ fix comment + close → diary ledger

Players leave typed or **voice** notes right in the game. Each is de-duplicated and promoted to an `in-game-note` issue. The agent reproduces, fixes, **re-gates at 0 deaths**, posts a ✅ fix comment, and closes it. That comment becomes the diary entry — a programmatic "Notes → fixes ledger" served in-game at `/diary.html`.

The build diary holds **75 dated entries + 41 numbered engine-level learnings** across 161 KB. The voice-note loop filed **8 real issues within hours** of shipping.



The whole campaign now ships on Claystone

Rainbow Run — a full, months-grown Phaser game — was migrated onto Claystone and cut over. The migration validated every claim in this deck against real, shipped content.

0.37s	~15 min	25/25	16 / 130
TO GATE ALL 25 LEVELS	THE SAME WORK ON PHASER	BYTE-IDENTICAL ROUND-TRIP	SDK SURFACES / CALL SITES MAPPED

"What took ~15 browser-minutes on Phaser runs in about a second here — and each run is **bit-exact by construction**." Parity is proven route-for-route against recorded Phaser golden traces: same route, same outcome, 0 deaths, frames within ±10%.

WHERE THIS GOES

One engine. Many worlds.
A studio that improves itself.

Claystone makes games **reproducible**. The templates make them **fast**. The eval + feedback loops make them **get better on their own**. Rainbow Run proves all three at once — 25 worlds, gated in a bit-exact third of a second.

The line to keep moving is the one between **recipes an AI runs by hand** and **machines that run themselves the same way every time**. Claystone is the machine the whole studio now stands on.

Claystone 🧱 · game-engine · rainbow-run — built & AI-evaluated with Claude Code · art by Gemini Nano Banana Pro · 2026